

第086课答案与真实CPU运行核对

1 类与继承

一个最小写法如下。两个对象a和b各自保存value，修改a不自动修改b；如果属性指向同一个外部可变对象则另当别论，这也是理解张量共享存储的重要铺垫。

```
class Counter:
    def __init__(self, start=0):
        self.value = start
    def add(self, amount):
        self.value += amount
        return self.value

class NamedCounter(Counter):
    def __init__(self, name, start=0):
        super().__init__(start)
        self.name = name

a = NamedCounter('训练', 0)
b = NamedCounter('验证', 0)
a.add(3)
print(a.value, b.value)  # 3 0
```

self是当前实例；super让继承来的初始化先建立value，再添加name。nn.Module中的基类初始化还承担参数/子模块登记等框架职责，不能把这个简单Counter的行为类推成“所有父类初始化都只是设置一个数值”。

2 生成器与epoch

```
def mini_batches(values, size):
    for start in range(0, len(values), size):
        yield values[start:start + size]
```

调用mini_batches得到一个生成器，next每次从上一次yield处继续。遍历结束后状态已经到函数末尾，再对同一个生成器迭代不会重新执行。新epoch应重新调用mini_batches或重新遍历能够产生新迭代器的DataLoader，而不是重复使用旧的耗尽对象。

生成器的优势是逐步提供元素，不必事先保存所有批量；它并不自动打乱或复制数据。切片是否复制还取决于底层类型，NumPy切片经常是视图。

3 参数布局

PyTorch第一层weight为 3×2 ，第二层weight为 2×3 ；它们分别等于NumPy W1.T与W2.T。bias为3项和2项，直接复制。导出到NumPy约定时，weight再转置一次；weight.grad也必须转置后才与G_W1、G_W2比较，bias.grad不转置。

如果只在导入参数时转置，比较梯度时不转置，看到的差异是布局错配，不是autograd算错。总元素数为 $6+3+6+2=17$ ，与两种存储方式无关。

4 梯度累积

$L = (3w - 1)^2$ ，所以 $L' = 2(3w - 1) \times 3$ ， $w=2$ 时为30。参数不变、每次重新前向，第一次grad=30，第二次不清空则grad=60；设为None后第三次反向又得到30。

如果中间以学习率0.1更新， w 变成-1；下一次局部梯度为 $2(-4) \times 3 = -24$ 。若旧30仍留着，累积结果会是6，而不是60。这说明忘记清梯度的问题不是固定的“两倍学习率”，不能总靠把学习率减半解决。

5 尾批加权

正确全体平均为 $(32 \times 0.2 + 3 \times 1.0)/(32 + 3) = 9.4/35 \approx 0.268571429$ 。0.6来自两个批均值再平均，让两个批量各占一半权重，因而每条尾批样本被赋予更大权重。

稳妥做法是累计每条损失的总和，最后除样本总数；或者用每批mean乘当批样本数再累加。本课两种等价方式分别用于训练与验证，并与固定最终模型的NumPy整体验证损失对照，实测绝对差约 $3.47\text{e-}18$ 。

6 eval与no_grad

eval改变模块训练状态，Dropout停止随机丢弃、BatchNorm等模块采用相应评价行为；它不会自动关闭autograd记录。no_grad控制记录，不自动把模块切到评价态。只有Linear和tanh时，漏eval可能暂时看不出预测差异，但一旦加入状态敏感层就会产生错误行为。

只用eval而没有no_grad，通常还能得到正确预测，但保留不必要的图与内存，且可能误执行反向；只用no_grad而没有eval，Dropout仍可随机化，BatchNorm还可能更新训练统计。本次实验Dropout训练零比例0.506，eval全部返回1；eval单独前向requires_grad为True，加no_grad后为False。

7 迁移验收

最低三层：比较logits和标量损失；比较四块全部参数梯度；比较相同SGD规则的一步新参数。需要相同数据与标签、相同参数数值、布局映射、dtype、device、激活、损失归约及优化器设置；随机层也要控制或关闭。

最终准确率把很多连续信息压成一个比例，错误实现也可能碰巧获得同样比例。先控制输入和参数，再逐层比较，才能定位迁移差异。本次真实CPU float64运行：损失两边都是0.6888129044681032；logits最大差1.1102230e-16；W1、b1、W2、b2梯度最大差分别约1.73e-17、3.47e-17、2.78e-17、5.55e-17；一步更新最大差约3.47e-18。

8 正确微批量累积

先清空一次梯度。第一微批量用sum损失除35后backward，第二微批量也用sum损失除35后backward，期间不step，最后统一step一次。因为求导是线性的，累积结果就是两批所有单样本梯度总和除35。

若每批先mean，再各除2，两个微批量各占一半，会与35条样本等权平均不同。要用mean也可以，分别乘32/35和3/35，再累积。对于带BatchNorm或随机层的网络，微批量前向本身还可能不同，所以这里的严格等价以前向逐样本独立且可控为前提。

9 检查点与数据职责

state_dict返回状态映射，其中张量可能仍关联模型存储。只把映射赋给best变量，不能保证后续训练不会改变被保存数值。深拷贝创建独立快照，之后恢复的才是验证最优时刻，而不是最终时刻的别名。

验证已用于选择检查点，不再是最终独立证据。测试只在选择结束、模型冻结后评价一次，不用于决定是否重试或挑选另一个种子。若根据测试修改方案，必须把这次结果视为开发信息，并另备保留评价。

冻结完整训练

环境：Python 3.12.14、NumPy 2.3.5、PyTorch 2.14.1+cpu，CPU float64，单计算线程。数据323/157/160，batch_size=32，训练尾批3、验证尾批29；180个epoch，每个epoch11次更新，SGD学习率0.15。第180个epoch的验证损失最低。

恢复后训练损失0.005283997、准确率1.0；验证损失0.013675806、准确率0.993630573；测试损失0.009206825、准确率1.0。普通及-O模式各11项unittest通过，未跳过PyTorch执行。统计数字允许机器精度级差异，不要求每个平台字节相同。

本次比085损失更低不构成框架优劣结论：两讲更新次数、批量大小与检查点策略不同。这是一份迁移正确性与训练管道示范，不是公平性能竞赛，也不是现实数据泛化保证。